

DSDL Lab 2

B13902022 Richard Lai

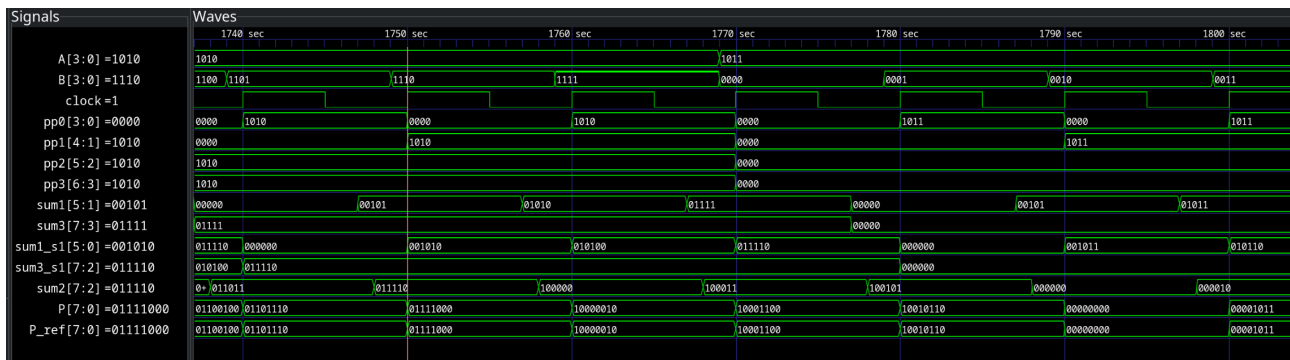
May 12, 2026

1 Part 1

1.1

```
module mult_fast(  
    output reg[7:0] P, // product  
    input[3:0] A, B, // multiplicand and multiplier  
    input clk // clock (posedge)  
);  
    // stage 0 (input)  
    reg[3:0] a_s0, b_s0;  
    always @(posedge clk) begin  
        a_s0 <= A;  
        b_s0 <= B;  
    end  
    // stage 1  
    wire[3:0] pp0 = a_s0 & {4{b_s0[0]}}; // ignore the delays of AND gates  
    wire[4:1] pp1 = a_s0 & {4{b_s0[1]}}; // ignore the delays of AND gates  
    wire[5:2] pp2 = a_s0 & {4{b_s0[2]}}; // ignore the delays of AND gates  
    wire[6:3] pp3 = a_s0 & {4{b_s0[3]}}; // ignore the delays of AND gates  
    reg[5:1] sum1;  
    always @(pp0, pp1)  
        sum1[5:1] <= #7 pp0[3:1] + pp1[4:1]; // delay of the 4-bit adder  
    reg[7:3] sum3;  
    always @(pp2, pp3)  
        sum3[7:3] <= #7 pp2[5:3] + pp3[6:3]; // delay of the 4-bit adder  
    reg[5:0] sum1_s1;  
    reg[7:2] sum3_s1;  
    always @(posedge clk) begin  
        sum1_s1 <= {sum1, pp0[0]};  
        sum3_s1 <= {sum3, pp2[2]};  
    end  
    // stage 2 (output)  
    reg[7:2] sum2;  
    always @(sum1_s1, sum3_s1)  
        sum2[7:2] <= #8 sum1_s1[5:2] + sum3_s1[7:2]; // delay of the 6-bit adder  
    always @(posedge clk) begin  
        P <= {sum2, sum1_s1[1:0]};  
    end  
endmodule
```

1.2



1.3

In the worst-case scenario, data arriving just after a clock edge must wait almost a full cycle to be sampled by Stage 0. After being sampled by the input register, the data passes through two additional pipeline stages before reaching the output register. The latency is 3 clock cycles in total. Given the default testbench clock cycle of 10 ticks, the total latency is **30 ticks**.

2 Part 2

2.1

In a pipelined architecture, the minimum clock cycle is constrained by the longest combinational delay between any two sequential registers (the critical path).

We can look at `mult_fast`: Stage 1 maximum delay is 7 ticks, and stage 2 maximum delay is 8 ticks.

Hence, the absolute minimum clock cycle to ensure timing requirements are met without a race condition is 8 ticks.

2.2

```
module mult_tb();
    // dump
    initial begin
        $dumpfile("lab2.vcd");
        $dumpvars(0, mult_tb);
    end
    // clock cycle = 8 ticks
    reg clock = 1;
    always
        #4 clock <= ~clock;
    // multiplier
    reg[3:0] A, B;
    wire[7:0] P;
    reg[7:0] P_ref;
    mult_fast mult(P, A, B, clock);
    always @(posedge clock)
        P_ref <= #16 A*B;
    // loop through all possible inputs
    integer i;
```

```

initial begin
    #7;
    for(i=0; i<256; i=i+1) begin
        {A, B} <= i;
        #8;
    end
    #17 $finish;
end
// check if the products are correct
reg[3:0] A_old, B_old;
always @(posedge clock) begin
    #1
    if(P_ref!=P && i>=2) begin
        A_old = (i-2)/16;
        B_old = (i-2)%16;
        $display("Product is wrong when A=%b, B=%b.", A_old, B_old);
        $display("P_ref: %b.", P_ref);
        $display("P      : %b.", P);
        #7 $finish;
    end
end
endmodule

```

